

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Вариант №9

по дисциплине «Методы оптимизации»

Обучающийся: Журбина Марина Андреевна

Поток: 1.2

Условие задачи:

Максимизировать $Z = 2x_1 + x_2 + 3x_3 + 2x_4$
при условиях:

$$x_1 + 2x_2 + x_3 \leq 11$$

$$x_1 + x_3 + x_4 = 8$$

$$x_2 + x_4 \geq 3$$

Ход ручного решения и ответ:

Вариант 9

Исходная ЗЛП:

Максимизировать: $Z = 2x_1 + x_2 + 3x_3 + 2x_4$

При условиях:

$$\begin{cases} x_1 + 2x_2 + x_3 \leq 11 \\ x_1 + x_3 + x_4 = 8 \\ x_2 + x_4 \geq 3 \\ x_1, x_2, x_3, x_4 \geq 0 \end{cases}$$

1) Приведение задачи к каноническому виду:

1. Изменяем целевую функцию на -1 , чтобы перейти от максимизации к минимизации:

$$W = -Z = -2x_1 - x_2 - 3x_3 - 2x_4 \rightarrow \min$$

2. Избавляемся от неравенств, путем добавления дополнительной переменной со знаком $+$ при знаке неравенства " $\leq$ " и вычитанием дополнительной переменной при знаке неравенства " $\geq$ ":

Канонический вид:

$$W = -2x_1 - x_2 - 3x_3 - 2x_4 \rightarrow \min$$

$$\begin{cases} x_1 + 2x_2 + x_3 + x_5 = 11 \\ x_1 + x_3 + x_4 = 8 \\ x_2 + x_4 - x_6 = 3 \\ x_1, x_2, x_3, x_4, x_5, x_6 \geq 0 \end{cases}$$

2) Формулирование вспомогательной задачи:

Чтобы начать решать задачу симплекс-методом, в каждой ур-ии должна быть базисная переменная с коэф. 1, которая встречается только в этой ур-ии. В 1 ур-ии это x_5 . Во 2 и 3 ур-ии такой базис нет. Поэтому вводим искусственные переменные $x_7, x_8 \geq 0$

$$\begin{cases} x_1 + 2x_2 + x_3 + x_5 = 11 \\ x_1 + x_3 + x_4 + x_7 = 8 \\ x_2 + x_4 - x_6 + x_8 = 3 \end{cases}$$

Целевая функция транспортной задачи: $W' = x_7 + x_8 \rightarrow \min$

Выразим искусств. переменные через остальные:

$$x_7 = 8 - x_1 - x_3 - x_4$$

$$x_8 = 3 - x_2 - x_4 + x_6$$

Подставим их в W' :

$$W' = (8 - x_1 - x_3 - x_4) + (3 - x_2 - x_4 + x_6) = 11 - x_1 - x_2 - x_3 - 2x_4 + x_6 \rightarrow \min$$

3) Решение транспортной задачи:

Составим начальную симплекс-таблицу. В строке Δ записываем коэфф. при переменных из функции W' .

Индекс x_{bas}	1	2	3	4	6	6
5	1	2	1	0	0	11
7	1	0	1	1	0	8
8	0	1	0	1	-1	3
Δ	-1	-1	-1	-2	1	-11

1. Выбор разрешающего столбца: x_4 (самый отрицательный $p = -2$)

2. Выбор разрешающей строки: строка переменных x_8 (потому что $\frac{8}{1} = 8$, $\frac{3}{1} = 3$, берём минимальное)

3. Разрешающий элемент: $\hat{a}_{84} = 1$. Переменные x_4 входят в базис, x_8 выходит.

Пересчитываем таблицу:

Индекс x_{bas}	1	2	3	6	6
5	1	2	1	0	11
7	1	-1	1	1	5
4	0	1	0	-1	3
Δ	-1	1	-1	-1	-5

1. Выбор разрешающего столбца: x_1 (самый маленький $p = -1$)

2. Выбор разрешающей строки: строка переменных x_7 (потому что $\frac{5}{1} = 5$)

$\frac{5}{1} = 5$, берем минимальное)

3. Разрешающий элемент: $\hat{a}_{41} = 1$. Переменная x_1 входит, x_3 выходит.
Искусственные переменные становятся нуль.

Индекс x^{bas}	2	3	6	6
5	3	0	-1	6
1	-1	1	1	5
4	1	0	-1	3
Δ	0	0	0	0

В строке Δ нет отрицательных чисел, $W' = 0$, значит вспомогательная задача решена.

4. Переход к основной задаче:

$$W = -2x_1 - x_2 - 3x_3 - 2x_4$$

Выразим базисные переменные (x_1, x_4) через свободные (x_2, x_3, x_6) :

$$x_1 = 5 + x_2 - x_3 - x_6$$

Подставляем в W :

$$W = -2(5 + x_2 - x_3 - x_6) - x_2 - 3x_3 - 2(3 - x_2 + x_6)$$

$$W = -x_2 - x_3 - 16 \rightarrow \min$$

5. Решение основной задачи

Составляем начальную таблицу для основной задачи:

Индекс x^{bas}	2	3	6	6
5	3	0	-1	6
1	-1	1	1	5
4	1	0	-1	3
Δ	-1	-1	0	16

1. Выбор разрешающей строки: x_2 (самый маленький $p = -1$)

2. Выбор разрешающей строки: строка переменной 5 (поэтому что

$\frac{6}{3} = 2, \frac{3}{1} = 3$, берём минимальное, но так как не делим)

3. Разрешающий элемент: $\hat{a}_{12} = 3$. Переменная x_2 входит, x_3 выходит.

Перестраиваем таблицу:

Индекс x_{bas}	5	3	6	6
2	$\frac{1}{3}$	0	$-\frac{1}{3}$	2
1	$\frac{1}{3}$	1	$\frac{2}{3}$	7
4	$-\frac{1}{3}$	0	$-\frac{2}{3}$	1
Δ	$\frac{1}{3}$	-1	$-\frac{1}{3}$	18

1. Выбор разреш. столбца: x_3 (самый левый $\rho = -1$)

2. Выбор разреш. строки: строка переменной x_1

3. Разрешающий элемент: $\hat{a}_{13} = 1$

Перестраиваем таблицу:

Индекс x_{bas}	5	1	6	6
2	$\frac{1}{3}$	0	$-\frac{1}{3}$	2
3	$\frac{1}{3}$	1	$\frac{2}{3}$	7
4	$-\frac{1}{3}$	0	$-\frac{2}{3}$	1
Δ	$\frac{2}{3}$	1	$\frac{1}{3}$	25

В строке Δ нет отрицательных чисел, значит задача решена.

Значения базисных переменных берём из столбца 6, не входящие в базис переменные равны 0.

Оптимальная точка: $x^* = (x_1, x_2, x_3, x_4) = (0, 2, 7, 1)$

Значение целевой функции в оптимальной точке:

$$Z_{\max} = -W = -(-25) = 25.$$

Ответ: $x^* = (0, 2, 7, 1)$
 $Z = 25$

Ответ:

Оптимальная точка:

$$x^* = (x_1, x_2, x_3, x_4) = (0; 2; 7; 1)$$

Значение целевой функции в оптимальной точке:

$$Z = 25$$

Ход решения в Excel:

В ходе проверки решения в Excel (надстройка Solver) были заданы целевая функция и ограничения. В ячейках A1-E1 я вписала текстом переменные и функцию. В ячейке E2 я вписала функцию $= 2 * A2 + B2 + 3 * C2 + 2 * D2$. Ниже в ячейках A6-A8 я вписала неравенства: $A6 = A2 + 2 * B2 + C2$; $A7 = A2 + C2 + D2$; $A8 = B2 + D2$. Далее я открыла поиск решения в эксель, задала ячейки переменных (A2-D2), целевую функцию (E2), добавила ограничения для ячеек A6-A8, выбрала все необходимые настройки и нажала «найти решение».

The screenshot shows the 'Параметры поиска решения' (Solver Parameters) dialog box in Excel. The 'Set Objective' field is set to '\$E\$2'. The 'To: Of' section has 'Maximum' selected. The 'Variable Cells' field is '\$A\$2:\$D\$2'. The 'Constraints' list contains three items: '\$A\$6 <= 11', '\$A\$7 = 8', and '\$A\$8 >= 3'. The 'Make Variable Non-Negative' checkbox is checked. The 'Select a GRG Nonlinear engine' dropdown is set to 'Поиск решения лин. задач симплекс-методом'. The 'Load/Save' button is visible. At the bottom, there are buttons for 'Справка', 'Найти решение', and 'Закрыть'.

Параметры поиска решения

Оптимизировать целевую функцию:

До: ☒ Максимум ☐ Минимум ☐ Значения:

Изменяя ячейки переменных:

В соответствии с ограничениями:

-
-
-

☒ Сделать переменные без ограничений неотрицательными

Выберите метод решения:

Метод решения

Для гладких нелинейных задач используйте поиск решения нелинейных задач методом ОПГ, для линейных задач - поиск решения линейных задач симплекс-методом, а для негладких задач - эволюционный поиск решения.

Добавить

Изменить

Удалить

Сбросить

Загрузить/сохранить

Параметры

Справка

Найти решение

Закрыть

Полученное Solver решение совпало с результатом ручного решения:

A	B	C	D	E
x1	x2	x3	x4	Z
0	2	7	1	25
11	<=	11		
8	=	8		
3	>=	3		

Программное решение:

```
import numpy as np
from fractions import Fraction

class SimplexSolver:
    def __init__(self, objective, constraints, b, signs, maximize=True):
        self.orig_obj = [Fraction(x) for x in objective]
        self.maximize = maximize
        self.constraints = [[Fraction(x) for x in row] for row in constraints]
        self.b = [Fraction(x) for x in b]
        self.signs = signs

        self.num_orig_vars = len(objective)
        self.num_constraints = len(b)

        self.tableau = None
        self.basis = []
        self.artificial_vars = []
        self.var_names = []

    def print_tableau(self, step_name):
        print(f"\n--- {step_name} ---")
        header = "Базис | " + " | ".join(f"{name:>5}" for name in self.var_names) + " | b"
        print("-" * len(header))
        print(header)
        print("-" * len(header))

        for i in range(self.num_constraints):
            bas_name = self.var_names[self.basis[i]]
            row_str = " | ".join(f"{str(x):>5}" for x in self.tableau[i, :-1])
            print(f"{bas_name:>3} | {row_str} | {str(self.tableau[i, -1]):>5}")

        print("-" * len(header))
        delta_str = " | ".join(f"{str(x):>5}" for x in self.tableau[-1, :-1])
        print(f"Δ | {delta_str} | {str(self.tableau[-1, -1]):>5}")
        print("-" * len(header))

    def setup(self):
        # Приведение к каноническому виду
        if self.maximize:
            self.c = [-x for x in self.orig_obj]
        else:
            self.c = list(self.orig_obj)

        self.var_names = [f"x{i + 1}" for i in range(self.num_orig_vars)]

        # Добавляем дополнительные и искусственные переменные
        slacks = []
        artificials = []
```

```

for i in range(self.num_constraints):
    if self.b[i] < 0:
        self.b[i] = -self.b[i]
        self.constraints[i] = [-x for x in self.constraints[i]]
        self.signs[i] = '>=' if self.signs[i] == '<=' else '<=' if self.signs[i] == '>=' else '='

    if self.signs[i] == '<=':
        slacks.append((i, 1))
        self.basis.append(len(self.var_names))
        self.var_names.append(f'x {len(self.var_names) + 1}')
    elif self.signs[i] == '>=':
        slacks.append((i, -1))
        self.var_names.append(f'x {len(self.var_names) + 1}')

    artificials.append(i)
    self.basis.append(len(self.var_names))
    self.artificial_vars.append(len(self.var_names))
    self.var_names.append(f'x {len(self.var_names) + 1}')
    elif self.signs[i] == '=':
        artificials.append(i)
        self.basis.append(len(self.var_names))
        self.artificial_vars.append(len(self.var_names))
        self.var_names.append(f'x {len(self.var_names) + 1}')

# Формируем матрицу
num_vars = len(self.var_names)
self.tableau = np.zeros((self.num_constraints + 1, num_vars + 1), dtype=object)

# Заполняем основные переменные
for i in range(self.num_constraints):
    self.tableau[i, :self.num_orig_vars] = self.constraints[i]
    self.tableau[i, -1] = self.b[i]

# Заполняем дополнительные и искусственные переменные
for idx, (row, val) in enumerate(slacks):
    col = self.num_orig_vars + idx
    self.tableau[row, col] = Fraction(val)

for idx, row in enumerate(artificials):
    col = self.num_orig_vars + len(slacks) + idx
    self.tableau[row, col] = Fraction(1)

def pivot(self, pivot_row, pivot_col):
    # Пересчитываем таблицу
    pivot_val = self.tableau[pivot_row, pivot_col]
    self.tableau[pivot_row, :] /= pivot_val

    for i in range(len(self.tableau)):
        if i != pivot_row:
            self.tableau[i, :] -= self.tableau[i, pivot_col] * self.tableau[pivot_row, :]

    self.basis[pivot_row] = pivot_col

def solve_phase(self, phase_name):
    iteration = 1
    while True:
        self.print_tableau(f'{phase_name} - Итерация {iteration}')
        delta_row = self.tableau[-1, :-1]

        # Ищем разрешающий столбец
        min_val = min(delta_row)
        if min_val >= 0:

```

```

        print("Отрицательных элементов в  $\Delta$  нет. Фаза завершена.")
        break

    pivot_col = np.argmin(delta_row)

    # Ищем разрешающую строку
    ratios = []
    for i in range(self.num_constraints):
        val = self.tableau[i, pivot_col]
        if val > 0:
            ratios.append(self.tableau[i, -1] / val)
        else:
            ratios.append(float('inf'))

    min_ratio = min(ratios)
    if min_ratio == float('inf'):
        raise Exception("Решение не ограничено (целевая функция уходит в бесконечность)")

    pivot_row = ratios.index(min_ratio)
    print(f'Входит в базис: переменная {self.var_names[pivot_col]}, Выходит: переменная {self.var_names[self.basis[pivot_row]]}')
    print(f'Разрешающий элемент: {self.tableau[pivot_row, pivot_col]}')

    # Пересчет таблицы
    self.pivot(pivot_row, pivot_col)
    iteration += 1

def solve(self):
    self.setup()

    # Вспомогательная задача
    if self.artificial_vars:
        print("\nРешение вспомогательной задачи:")
        # Целевая функция: сумма искусственных переменных
        for art_col in self.artificial_vars:
            self.tableau[-1, art_col] = Fraction(1)

    # Подстановка
    for i in range(self.num_constraints):
        bas_col = self.basis[i]
        if self.tableau[-1, bas_col] != 0:
            self.tableau[-1, :] -= self.tableau[i, :] * self.tableau[-1, bas_col]

    self.solve_phase("Фаза 1")

    if self.tableau[-1, -1] < 0: # С учетом знака Q
        raise Exception("Система ограничений несовместна (пустое множество решений)")

    # Удаляем столбцы искусственных переменных
    cols_to_keep = [i for i in range(len(self.var_names)) if i not in self.artificial_vars] + [-1]
    self.tableau = self.tableau[:, cols_to_keep]
    self.var_names = [name for i, name in enumerate(self.var_names) if i not in self.artificial_vars]

    # Обновляем индексы базиса
    for i in range(len(self.basis)):
        if self.basis[i] in self.artificial_vars:
            pass # Теоретически сюда не дойдем, если решение есть
        else:
            # Сдвигаем индекс из-за удаленных столбцов
            self.basis[i] -= sum(1 for art in self.artificial_vars if art < self.basis[i])

    # Основная задача
    print("\nРешение основной задачи:")

```

```

self.tableau[-1, :] = 0 # Очищаем строку Δ

# Восстанавливаем целевую функцию
for i in range(self.num_orig_vars):
    self.tableau[-1, i] = self.c[i]

# Подстановка базисных переменных (выражение через свободные)
for i in range(self.num_constraints):
    bas_col = self.basis[i]
    if self.tableau[-1, bas_col] != 0:
        self.tableau[-1, :] -= self.tableau[i, :] * self.tableau[-1, bas_col]

self.solve_phase("Фаза 2")

# Вывод ответа
print("\nОтвет:")
ans = [Fraction(0)] * self.num_orig_vars
for i in range(self.num_constraints):
    if self.basis[i] < self.num_orig_vars:
        ans[self.basis[i]] = self.tableau[i, -1]

ans_str = ", ".join(str(x) for x in ans)
print(f"Оптимальная точка X* = ({ans_str})")

z = self.tableau[-1, -1] if not self.maximize else -self.tableau[-1, -1]
print(f"Значение целевой функции Z = {z}")

# Задача из Варианта №9
if __name__ == "__main__":
    # Целевая функция
    objective = [2, 1, 3, 2] # Коэффициенты при x1, x2, x3, x4

    # Коэффициенты при переменных в левой части условий
    constraints = [
        [1, 2, 1, 0], # x1 + 2x2 + x3 ≤ 11
        [1, 0, 1, 1], # x1 + x3 + x4 = 8
        [0, 1, 0, 1] # x2 + x4 ≥ 3
    ]

    # Свободные члены условий
    b = [11, 8, 3]

    # Знаки условий
    signs = ['<=', '=', '>=']

    # Создаем и запускаем решатель
    solver = SimplexSolver(objective, constraints, b, signs, maximize=True)
    solver.solve()

```

Рефлексивный вывод:

Работа была полезной, потому что я на практике разобралась, как составлять задачу линейного программирования, приводить её к каноническому виду и решать симплекс-методом. Стало понятнее, зачем нужна вспомогательная задача и как по шагам заполняется симплекс-таблица.

Самым сложным для меня было понять, как выбирать разрешающий столбец и строку, почему в отношениях учитываются только положительные элементы. Я справилась с этим, когда начала строго идти по алгоритму из методички и внимательно проверять вычисления после каждого шага. Проверка в Excel показала тот же ответ, поэтому я убедилась, что решение выполнено правильно.